

Problem Description

You are given a weighted graph with **N** vertices and **M** edges. Find the total weight of its **maximum** spanning tree.

Constraints

- $1 \leq T \leq 20$
- $1 \leq N \leq 5000$
- $1 \leq M \leq 100\,000$
- $1 \leq c \leq 10\,000$

Input

The first line contains one integer **T** denoting the number of test cases. Each test case starts with a line containing **2** space-separated integers: **N** and **M**. Each of the following **M** lines contain description of one edge: three different space-separated integers: **a**, **b** and **c**. **a** and **b** are different and from 1 to **N** each and denote numbers of vertices that are connected by this edge. **c** denotes weight of this edge.

Output

For each test case output one integer - the total weight of its **maximum** spanning tree.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

1
3 3
1 2 2
2 3 3
1 3 4

EXPECTED OUTPUT

7

Test Case 2

INPUT (STDIN)

1
3 3
1 2 3
2 3 4
1 3 5

EXPECTED OUTPUT

9

▼ Mandatory Test Cases

Test Case 1

KEYWORD

int printheap(int N)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

715

Test Case 3

NLOC

110

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 typedef struct {
6     int u, v, w;
7 } Edge;
8
9 int cmp_edge(const void *a, const void *b) {
10     Edge *e1 = (Edge*)a;
11     Edge *e2 = (Edge*)b;
12     return e2->w - e1->w;
13}
14
15int find_set(int *parent, int i) {
16     if (parent[i] == i)
17         return i;
18     return parent[i] = find_set(parent, parent[i]);
19}
20
21void union_sets(int *parent, int *rank, int i, int j) {
22     int root_i = find_set(parent, i);
23     int root_j = find_set(parent, j);
24     if (root_i != root_j) {
25         if (rank[root_i] < rank[root_j]) {
```

```
26     parent[root_i] = root_j;
27     } else if (rank[root_i] > rank[root_j]) {
28         parent[root_j] = root_i;
29     } else {
30         parent[root_j] = root_i;
31         rank[root_i]++;
32     }
33 }
34}
35
36int printheap(int N) {
37     return N;
38}
39
```

```
int main() {
```

main | CH.SC.U4CSE25225 | Tree24.c

Ln 80, Col 1 Spaces: 4 UTF-8 CRLF C

```
42     if (scanf("%d", &t) != 1) return 0;
43     while (t--) {
44         int n, m;
45         if (scanf("%d %d", &n, &m) != 2) break;
46         Edge *edges = (Edge*)malloc(m * sizeof(Edge));
47         for (int i = 0; i < m; i++) {
48             if (scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w) != 3) {
49                 edges[i].u = edges[i].v = edges[i].w = 0;
50             }
51         }
52
53         qsort(edges, m, sizeof(Edge), cmp_edge);
```

```
54
55     int *parent = (int*)malloc((n + 1) * sizeof(int));
56     int *rank = (int*)calloc(n + 1, sizeof(int));
57     for (int i = 1; i <= n; i++) parent[i] = i;
58
59     long long total_weight = 0;
60     int edges_count = 0;
61
62     for (int i = 0; i < m && edges_count < n - 1; i++) {
63         int u = edges[i].u;
64         int v = edges[i].v;
65         if (find_set(parent, u) != find_set(parent, v)) {
66             union_sets(parent, rank, u, v);
67             total_weight += edges[i].w;
```

```
68         edges_count++;
69     }
70 }
71
72     printf("%lld\n", total_weight);
73
74     free(edges);
75     free(parent);
76     free(rank);
77 }
78 return 0;
79 }
80
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

1
3 3
1 2 2
2 3 3
1 3 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree24.exe

7

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

1
3 3
1 2 3
2 3 4
1 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree24.exe

9

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>